

LeRobot SO-101 을 활용한 지능형 로봇팔 제어 : Sim2Real 및 잔차 강화학습 기반 정책 개선

2026 전기 Physical AI 졸업과제 최종보고서



팀명

키자니아패밀리

팀원

학번	이름	이메일
202355561	여채언	codjs2659@pusan.ac.kr
202355536	문여원	myw0422@pusan.ac.kr
202355562	옥소미	osm0071@pusan.ac.kr

지도교수 : 김원석

분과 : C

목차

1 서론.....	1
1.1 연구 배경 및 필요성.....	1
1.2 과제 목표.....	1
2 요구조건	3
2.1 초기 요구조건 대비 변경된 요구조건	3
2.2 착수 및 중간 설계에서 최종 구현으로의 변경	3
2.3 제약사항 및 대응 방안	4
3 연구 내용	7
3.1 시스템 설계	7
3.2 데이터 설계	11
3.3 MuJoCo 시뮬레이션 환경 설계	12
3.4 도메인 랜덤화 설계	13
3.5 IK 솔버 기반 스크립트 교사	15
3.6 잔차 강화학습 설계	17
3.7 Human-in-the-Loop 교정 데이터 수집 설계	24
4 연구 결과 분석 및 평가	25
4.1 시뮬레이션 데이터 개선 과정	25
4.2 학습 데이터 구성 비교	28
4.3 ACT와 SmolVLA 정책 비교	29
4.4 잔차 강화학습 실험 결과.....	29
4.5 최종 실물 추론 설정 탐색	33
4.6 최종 실물 평가 결과.....	34
5 결론 및 향후 연구 방향	36
5.1 성과	36
5.2 미완료 항목과 후속 개발.....	37

5.3 향후 연구 방향	38
6 과제 수행 관리	39
6.1 구성원별 역할과 수행 결과	39
6.2 개발 일정과 계획 대비 수행 결과	39
7 참고문헌	41

1 서론

본 과제는 SO-101 로봇팔의 시각 관측부터 데이터 생성, 정책 학습, 서버 추론, 실물 제어까지의 전체 과정을 구축하고 블록 이동 및 적재 성능을 평가하였다. 최종 혼합 데이터 ACT 정책은 Pick & Place 20회에서 평균 3.4개를 배치하고 5개 전체 성공률 25%를 기록하였다.

Pick & Stack은 사람 보조가 포함된 초기 평가 단계이며, 단일 모델의 두 미션 통합과 잔차 강화학습의 안정적 성능 개선은 후속 과제로 남았다. 본 보고서는 착수 계획, 중간 수정, 최종 구현과 실험 결과를 연결하여 기술한다.

1.1 연구 배경 및 필요성

Physical AI는 로봇이나 자율주행 시스템이 실제 환경을 인식하고, 이를 바탕으로 판단과 행동을 수행하는 기술이다. 기존의 로봇이 사전에 정의된 동작을 반복하는 방식에 가까웠다면, Physical AI 기반 로봇은 이미지와 센서 데이터를 이용해 주변 상태를 파악하고 상황에 따라 동작을 결정할 수 있다.

본 졸업과제에서는 이러한 개념을 LeRobot SO-101 로봇팔에 적용하여 블록 조작 과제를 수행한다. 종목 2인 「LeRobot SO-101을 활용한 지능형 로봇팔 제어」는 테이블 위에 임의로 배치된 블록을 집어 지정된 영역으로 옮기는 Pick & Place와, 블록을 안정적으로 쌓는 Pick & Stack을 목표로 한다.

블록의 위치와 자세가 매 시행마다 달라지기 때문에 미리 정해진 경로를 그대로 재생하는 방식만으로는 과제를 수행하기 어렵다. 따라서 카메라를 통해 현재 상태를 관찰하고, 학습된 정책이 이에 맞는 행동을 생성하여 로봇팔을 제어하는 과정이 필요하다.

본 과제에서는 비전 기반 관찰, 정책 학습, 로봇 제어를 하나의 시스템으로 구성하고, 다양한 초기 배치에서도 안정적으로 작업을 수행할 수 있는지 확인하는 것을 주요 목표로 한다.

1.2 과제 목표

과제는 난이도에 따라 두 개의 미션으로 구성된다. 아래 내용은 변경된 요구조건 v2.0을 기준으로 하며, 세부 변경 내역은 2장에서 설명한다.

표 1-1 미션별 요구 목표

구분	목표
1차 미션 place	경기장에 무작위로 배치된 서로 다른 색상의 블록 5개(4×4×2 cm)를 SO-101 로봇팔로 집어 지정 영역(20×10 cm) 안에 옮긴다. 제한 시간은 3분이며, 영역 안에 배치한 블록 수와 전체 성공 시 소요 시간을 기준으로 평가한다.
2차 미션 stack	블록 5개를 다시 초기 배치한 상태에서 시작하며, SO-101 로봇팔로 블록을 집어 지정 영역 안에 순차적으로 쌓는다. 제한 시간은 5분이며, 적재한 블록 수와 전체 성공 시 소요 시간을 기준으로 평가한다. 쌓은 블록이 5초 이상 안정적으로 유지되어야 성공으로 인정한다.
최종 목표	하나의 모델로 1차와 2차 미션을 모두 수행하며, 가능한 많은 블록을 정확하게 옮기거나 안정적으로 적재하고 수행 시간을 줄이는 것을 목표로 한다.

2 요구조건

2.1 초기 요구조건 대비 변경된 요구조건

표 2-1 초기 및 변경 요구조건 비교

항목	v1.5 (초기)	v2.0 (변경)
블록 크기	5×5×2 cm	4×4×2 cm
블록 준비	팀별 직접 준비	학과 배부 블록 사용 (5개 색상 상이)
초기 블록 위치	평가자가 무작위 결정	평가자가 무작위 결정, 모든 팀 동일
제한 시간	두 미션 통합 5분	1차 3분 / 2차 5분
1차 성공 기준	지정 영역 내 배치	선 안에 위치(선 걸침 2 cm까지 허용), 일부 겹쳐도 선 안이면 인정, 쌓기는 불인정
지정 영역	20×10 cm, 경계선 10/20 mm	내부 기준 20×10 cm, 경계선 최대 2 cm, 로봇 끝 기준 25 cm 지점 배치
경기장	A3 용지	배부된 체크보드 또는 A3 용지 위에 영역 표시
모델 사용 기기	제약 없음, 자유롭게	배부된 Jetson Orin Nano로 서버와 통신, 추론은 평가 서버에서 수행
카메라	높이·각도 자율	높이·각도 자율, 대수 제한 없음
신경망 모델 수	별도 규정 없음	두 task를 수행하더라도 신경망 모델은 하나여야 함. 규칙 기반 solution은 task별 복수 허용

마지막 항목은 정책 구조와 데이터 구성에 직접적인 영향을 주었다. 이에 두 미션을 별도 정책으로 분리하지 않고, 하나의 신경망이 task 조건을 입력받아 Pick & Place와 Pick & Stack을 모두 처리하도록 설계하였다.

2.2 착수 및 중간 설계에서 최종 구현으로의 변경

착수보고서는 실물 Tele-operation으로 SmoIVLA를 미세조정하고, Isaac Sim 기반 데이터 확장과 강화학습으로 성능을 높이는 단계적 계획을 제시하였다. 중간보고서에서는 MuJoCo 기본 경기장과 비동기 추론 환경을 구축하고, ACT와 SmoIVLA 외에 Diffusion Policy까지 비교 대상으로 검토하였다.

최종 보고에서는 완료된 ACT·SmolVLA 비교만 성능 근거로 사용한다. Diffusion Policy의 최종 정량 결과는 제공되지 않았다.

표 2-2 착수 계획과 중간 및 최종 구현의 변화

항목	착수 계획	중간 수행 및 검토	최종 구현과 결과
정책	SmolVLA 미세조정	ACT·SmolVLA·Diffusion Policy 비교 검토	ACT·SmolVLA 20회씩 비교 후 ACT 선정
시뮬레이션	Isaac Sim 활용 계획	MuJoCo 기본 환경 구축	MuJoCo 정합 보정, IK 교사, 179축 랜덤화
카메라	Wrist·Top·Side 3대	2대 조합 실험과 낮은 Top 각도 절충	정면 상단 사선 Side와 Wrist 2대 사용
추론 배포	Orin 연산 가능성 검토 후 서버 분리안 채택	RTX 4090 비동기 추론 구현	직접 통신 경로 및 JPEG 전송 적용
실패 대응	재시도와 RL 개선 계획	HIL 교정 수집과 랜덤화 적용 계획	HIL 제어권 전환·연속 기록 구현, 시뮬 교사 복구 경로 추가
단일 정책	미션별 시연 수집과 확장	task 조건 입력 필요성 검토	관측 task 확장 및 600개 통합 데이터 확보, 학습·평가 미완료

카메라 명칭은 문서 작성 시점에 따라 달랐다. 중간보고서는 낮은 각도의 Top과 Wrist를 절충안으로 기록했고, 최종 원고는 정면 상단에서 비스듬히 내려다보는 카메라를 Side로 명명한다. 본문은 최종 원고의 명칭을 사용하며, 두 기록이 완전히 같은 설치 자세라는 의미로 해석하지 않는다.

2.3 제약사항 및 대응 방안

(1) 제약 1. Orin Nano에서의 정책 추론 불가 → 서버 추론 구조 채택

Jetson Orin Nano를 통신 용도로만 사용하고, 정책 추론은 별도의 평가 서버에서 수행하도록 규정하고 있다. 여기에 Orin Nano에서 사용 가능한 CUDA·PyTorch 환경과 최신 LeRobot 제어 환경의 Python 버전 차이까지 겹치면서, 추론과 로봇 제어를 하나의 실행 환경에 구성하기 어려웠다.

이에 LeRobot의 Async Inference 구조인 RobotClient-PolicyServer 방식을 적용하여 로봇 제어와 정책 추론을 분리하였다.

표 2-3 추론과 로봇 제어의 장비별 역할

역할	장비	담당 작업
RobotClient	Jetson Orin Nano	관측 송신, action chunk 수신 및 로봇 제어
PolicyServer	RTX 4090 PC (평가 서버)	ACT 정책 추론

이 구조에서 Orin Nano는 관측값과 action을 중계하는 통신 장치이자 로봇 제어 장치로 동작하며, **정책 추론에는 참여하지 않는다**. 따라서 Orin에는 Python 3.12 기반의 LeRobot 제어 환경만 구성하여 추론 환경과의 버전 충돌을 피하였다.

(2) 제약 2. 서버-엣지 간 네트워크 통신 지연 및 안정성

초기에는 로봇에 탑재된 Orin Nano 에서 정책 추론을 직접 수행하고자 하였다. 그러나 엣지 장치의 제한된 연산 성능으로 인해 원하는 제어 속도를 안정적으로 확보하기 어려웠다. 이에 정책 추론을 별도의 GPU 서버에서 수행하는 구조로 변경하였다.

서버 추론 구조에서는 로봇이 카메라 관측값을 서버로 전송하고, 서버가 계산한 동작 명령을 다시 받아야 한다. 한 번에 전송하는 관측 데이터는(640 x 480) 크기의 RGB 이미지 두 장으로, 압축하지 않은 상태에서 약 1.84MB 이다. Orin Nano 의 무선 네트워크와 Tailscale 을 통해 이를 전송했을 때 전송 시간의 중앙값은 약 290ms 였으며, 실제 전송 속도는 약 50Mbps 였다. 로봇이 관측값을 보낼 때마다 서버의 계산 결과를 기다리면 동작이 중간에 멈추거나 제어 간격이 불규칙해질 수 있었다.

이러한 문제를 줄이기 위해 LeRobot 의 비동기 추론 방식을 적용하였다. 로봇은 관측값을 서버로 전송하고 서버가 새로운 동작을 계산하는 동안, 이전에 받아 둔 여러 개의 동작 명령을 계속 실행한다. 이를 통해 서버의 응답을 매 제어 주기마다 기다리지 않고 네트워크 통신, 서버 추론 및 로봇 제어를 동시에 진행할 수 있었다.

초기에는 정책 서버에 외부에서 직접 접근할 수 있는 통신 포트가 없어 Tailscale을 사용하였다. Tailscale은 서로 다른 네트워크에 있는 장치를 가상의 사설망으로 연결하는 오버레이 네트워크로, 별도의 공인 IP 설정 없이 장치 간 통신을 구성할 수 있다.

다만 두 장치가 직접 연결되지 않는 경우 중계 서버를 거쳐 패킷이 전달될 수 있으며, 이 경우 관측값이 Orin Nano에서 서버까지 전달되고 응답 action이 다시 돌아오는 과정에서 추가적인 지연이 발생한다.

이후 정책 서버에 RobotClient-PolicyServer 통신을 위한 포트를 요청하여 직접 연결할 수 있도록 변경하였다. 이에 따라 Tailscale의 중계 경로를 거치지 않고 Orin Nano와 정책 서버가 직접 통신하도록 구성하였으며, 네트워크 지연을 줄일 수 있었다.

마지막으로 전송해야 하는 데이터 자체를 줄이기 위해 이미지에만 **JPEG 압축**을 적용하였다. 용량이 큰 카메라 영상은 압축하여 전송하고, 관절 상태와 task 문자열처럼 크기가 작은 데이터는 별도 압축 없이 그대로 전달하도록 구성하였다. 이를 통해 통신량을 줄이면서도 로봇 제어에 필요한 상태 정보는 그대로 유지하였다.

(3) 제약 3. 팔로워 암 관절 유격에 따른 정밀도 저하

반복적인 데이터 수집 과정에서 관절 체결 나사가 풀리면서 유격이 발생했고, 이로 인해 Leader Arm에서 동일한 명령을 주더라도 Follower Arm의 말단 위치가 달라지는 문제가 나타났다. 이를 줄이기 위해 물리적으로 체결 나사를 접착제로 고정하고, 데이터 수집 전 관절 상태를 점검하는 절차를 추가하였다.

학습 단계에서는 남아 있을 수 있는 기구 오차를 도메인 랜덤화 항목에 포함하였다. 조인트 유격, 조립 공차, 관절 영점 오차를 랜덤화하여, 특정한 기구 상태에만 맞는 정책이 아니라 일정 수준의 오차가 존재하는 상황에서도 동작할 수 있도록 학습 환경을 구성하였다.

(4) 제약 4. 신경망 모델 하나로 두 미션

두 미션에 서로 다른 신경망을 사용할 경우 감점이 있으므로, 하나의 정책이 Pick & Place와 Pick & Stack을 모두 수행할 수 있어야 한다.

기존에 사용한 LeRobot ACT 정책은 데이터셋에 task_id가 포함되어 있더라도 이를 정책의 입력으로 사용하지 않는다. 즉, place와 stack 데이터를 함께 학습할 수는 있지만 추론 시 task 정보를 별도로 입력하여 현재 수행할 미션을 구분하는 구조는 아니다. VLA처럼 언어 명령이나 task 조건을 입력으로 받아 행동을 달리하는 방식과는 차이가 있다.

task 조건을 명시적으로 입력받는 단일 정책을 설계하였다. Pick & Place는 [1, 0], Pick & Stack은 [0, 1]로 표현한다. 최종 구현에서는 task one-hot을 관절 상태에 추가하여 observation.state를 6차원에서 **8차원으로 확장**하였다. 실물 로봇에 전달하는 **관절 목표는 6차원**이다.

따라서 최종 멀티태스크 상태 입력은 6차원 관절 상태 task one-hot을 포함한 8차원으로 확정하였다. 통합 데이터셋과 수집 래퍼는 마련하였으나, 하나의 학습 완료 모델이 두 미션을 모두 수행한다는 평가는 아직 완료되지 않았다.

3 연구 내용

3.1 시스템 설계

3.1.1 전체 시스템 구성

시스템은 **실물 장치**, **엣지 제어기**, **연산 서버**, **오프라인 시뮬레이션 환경**의 네 영역으로 구성하였다. MuJoCo는 실물 시스템에 포함하지 않고, 데이터 생성과 정책 검증을 담당하는 별도의 오프라인 환경으로 분리하였다.

표 3-1 시스템 구성과 데이터 흐름

영역	구성 요소	역할	주요 입출력 / 연결 방식
실물 장치	SO-101 Follower, SO-101 Leader, Side/Wrist 카메라	시연 데이터 수집 및 자율 동작 수행	관절 상태·명령, 640×480 RGB 영 상
엣지 제어	Jetson Orin Nano	관측 취득, 서버 전송, action queue 관리, 30 Hz 관절 명령 실행	카메라·관절 관측 입력, gRPC를 통 한 관측 송신 및 action chunk 수 신
연산 서버	RTX 4090 PC	정책 로딩 및 action chunk 추론	gRPC를 통한 관측 수신 및 action chunk 반환
오프라인 환 경	MuJoCo, 공유 GPU 서버	교사 궤적 생성, 도메인 랜 덤화, 정책 학습 및 페루프 평가	LeRobot Dataset v3.0 형식의 학습 데이터, 정책 체크포인트

Jetson Orin Nano와 RTX 4090 서버 사이의 통신에는 gRPC를 사용하였다. Orin은 카메라 영상과 로봇 상태를 서버로 전송하고, 서버는 정책 추론 결과인 action chunk를 다시 Orin으로 전달한다. 이를 통해 로봇 제어와 정책 추론을 서로 다른 장비에서 분리하여 수행할 수 있도록 구성하였다.

실물 환경과 시뮬레이션 환경에서는 동일한 LeRobot Dataset 형식을 사용하였다. 실물에서 수집한 tele-operation 데이터와 MuJoCo에서 생성한 시뮬레이션 데이터를 같은 관측·행동 스키마로 변환하여, 동일한 학습 파이프라인에서 사용할 수 있도록 구성하였다.

실물 데이터는 Leader-Follower 방식의 tele-operation으로 수집하였다. 각 프레임에는 Side/Wrist 카메라 영상, 로봇팔 5개 관절과 그리퍼의 상태, 실제 실행된 action, task 정보가 함께 기록된다.

시뮬레이션에서는 capstone_arena.episodes.Generator가 블록의 상태를 직접 참조하는 IK 기반 교사 정책으로 궤적을 생성하고, capstone_arena.to_lerobot이 이를 LeRobot Dataset 형식으로 변환한다.

학습 단계에서는 정책 후보로 ACT와 SmolVLA를 비교하였으며, 실험 결과를 바탕으로 ACT를 기준 정책으로 선택하였다. 이후 residual_rl을 별도로 구현하여 기준 정책의 행동을 제한된 범위에서 보정하고 성능을 개선할 수 있는지 검증하였다.

실물 배포 시에는 Jetson Orin Nano가 최신 관측값을 Policy Server로 전송하고, 서버는 추론한 action chunk를 반환한다. Orin은 전달받은 chunk를 30 Hz로 실행하면서 다음 추론 요청을 비동기적으로 전송한다. 관측 업로드와 데이터 기록은 제어 루프와 분리된 워커 스레드에서 처리하여 네트워크 통신이나 저장 작업이 로봇 제어 주기를 직접 지연시키지 않도록 구성하였다.

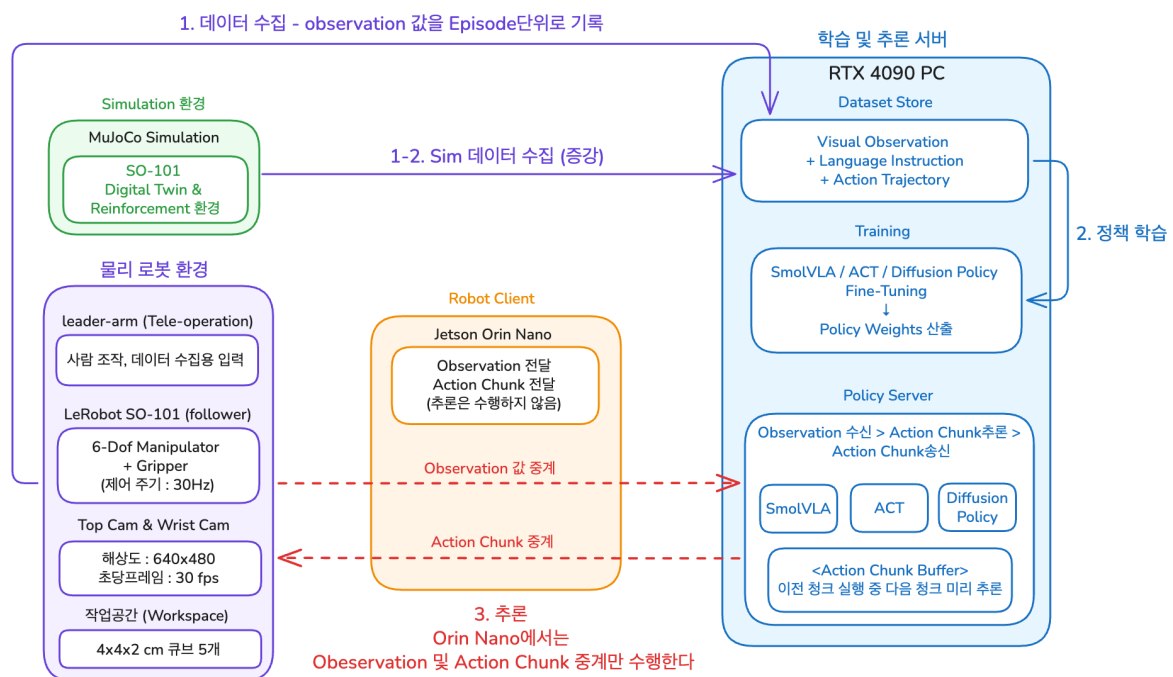


그림 3-1 데이터 수집부터 학습과 실물 제어까지의 흐름

3.1.2 하드웨어 구성

표 3-2 하드웨어 사양과 배치

구분	항목	사양 / 설정	비고
매니퓰레이터	LeRobot SO-101 (Follower)	팔 관절 5축 + 그리퍼 1축, 30 Hz 제어	6차원 action을 실행하는 실제 태스크 수행 장치

구분	항목	사양 / 설정	비고
입력 장치	Tele-operation Leader	Leader-Follower 관절 대응 매핑	시연 데이터 수집용
비전	Wrist Camera	640×480 / 30 fps, /dev/video2	그리퍼 근접 시점
비전	Side Camera	640×480 / 30 fps, /dev/video0	경기장 정면 상단에서 비스듬히 내려다보는 배치
Edge	Jetson Orin Nano	관측 송신, action 수신 및 로봇 제어	정책 추론은 수행하지 않음
Compute	RTX 4090 PC	VRAM 24 GB	Policy Server 구동
학습 서버	공유 GPU 서버	RTX 4090 ×8, 메모리 251 GB	데이터 생성, 정책 학습 및 평가
통신	Orin ↔ RTX 4090	Wi-Fi, gRPC	초기에는 Tailscale 사용, 실효 대역폭 약 50 Mbps 측정

SO-101의 6개 축에는 모두 동일한 STS3215 서보가 사용된다. 이에 따라 시뮬레이션에서도 관절별로 서로 다른 actuator parameter를 두지 않고, 여섯 관절에 동일한 파라미터를 적용하였다. 실물과의 차이는 액추에이터 및 관절 동역학 랜덤화를 통해 반영하였다.

3.1.3 소프트웨어 구성

표 3-3 소프트웨어 환경

구분	소프트웨어	역할
운영 환경	Ubuntu Linux, Python 3.12	실행 환경
로봇 프레임워크	Hugging Face LeRobot	Tele-operation, 데이터 기록, 정책 학습·평가
딥러닝	PyTorch ≥2.4, CUDA 12.8	학습 및 추론
영상 처리	OpenCV ≥5, <6	카메라 입력, 랜덤화 ISP 경로
시뮬레이션	MuJoCo ≥3.2	경기장 물리, 데이터 생성, 정책 평가
데이터 관리	LeRobot Dataset v3.0, Hugging Face Hub	영상·관절 상태·action-task

구분	소프트웨어	역할
학습 정책	ACT	최종 채택 base 정책
학습 정책	SmolVLA	비교 대상
비동기 추론	LeRobot Async Inference	추론과 제어 분리
강화학습	자체 구현 (residual RL)	모방학습 정책 개선 시도
영상 인코딩	libsvtav1 (AV1)	시물 데이터 저장
검증	pytest, 자체 검증 스크립트	회귀 및 경기장 계약 검사

3.1.4 시스템 동작 시나리오

시스템은 데이터 수집, 오프라인 학습, 비동기 추론, 평가의 순서로 동작한다.

1. 실물 데이터 수집

실물 환경에서는 Leader Arm으로 Follower Arm을 조작하여 시연 데이터를 수집한다. Side/Wrist 카메라 영상, 언어 지시, 관절 상태와 action을 시간 순서에 맞춰 하나의 에피소드로 저장한다.

2. 시뮬레이션 데이터 수집

시뮬레이션에서는 IK 기반 교사 정책이 도메인 랜덤화가 적용된 환경에서 rollout을 수행하며, 성공 기준을 만족한 에피소드만 선별하여 LeRobot Dataset 형식으로 변환한다.

3. 오프라인 학습

수집한 데이터셋을 이용해 base 정책을 학습하였다. ACT는 관측값으로부터 action sequence를 직접 예측하도록 학습하였고, SmolVLA는 사전학습된 모델을 기반으로 full fine-tuning을 수행하였다. 이 과정에서 vision encoder, VLM, action expert를 포함한 모델 구성 요소를 과제 데이터에 맞게 함께 학습하여 두 정책의 성능을 비교하였다.

4. 비동기 추론 및 배포

Jetson Orin Nano에서 수집한 관측값을 RTX 4090 서버로 전송하면, Policy Server가 action chunk를 추론하여 반환한다. Orin은 현재 전달받은 chunk를 30 Hz로 실행하는 동안 다음 chunk를 비동기적으로 요청한다. 이전 chunk와 새 chunk가 겹치는 구간은 aggregate 함수를 이용해 병합한다.

5. 종료 및 평가

미션 종료 후 성공 기준 충족 여부와 수행 시간을 기록하고, 반복 평가를 통해 성공률과 주요 실패 유형을 정리한다. 실패 사례는 촬영 영상과 실행 기록을 바탕으로 분류하며, 이후 데이터 보강과 정책 개선에 활용한다.

3.2 데이터 설계

실물 데이터와 시뮬레이션 데이터는 동일한 스키마를 사용하도록 구성하였다. 두 환경 모두 LeRobot Dataset v3.0 형식을 사용하며, 데이터 기록 주파수는 30 fps로 통일하였다.

표 3-4 개별 정책과 멀티태스크 데이터 스키마

키	개별 미션 정책	멀티태스크 확장
observation.images.side	480×640 RGB	동일
observation.images.wrist	480×640 RGB	동일
observation.state	관절 5축 + 그리퍼 1축의 6차원	관절 상태 6차원 + task one-hot 2차원
action	관절 및 그리퍼 목표 6차원	관절 및 그리퍼 목표 6차원

task는 Pick & Place [1, 0], Pick & Stack [0, 1]로 구분한다. 멀티태스크 확장에서는 task 정보를 action이 아닌 observation.state에 추가하였다. 본 보고서의 최종 실물 결과는 개별 미션 정책 평가이며, 통합 정책의 검증 결과와 구분한다.

3.3 MuJoCo 시뮬레이션 환경 설계

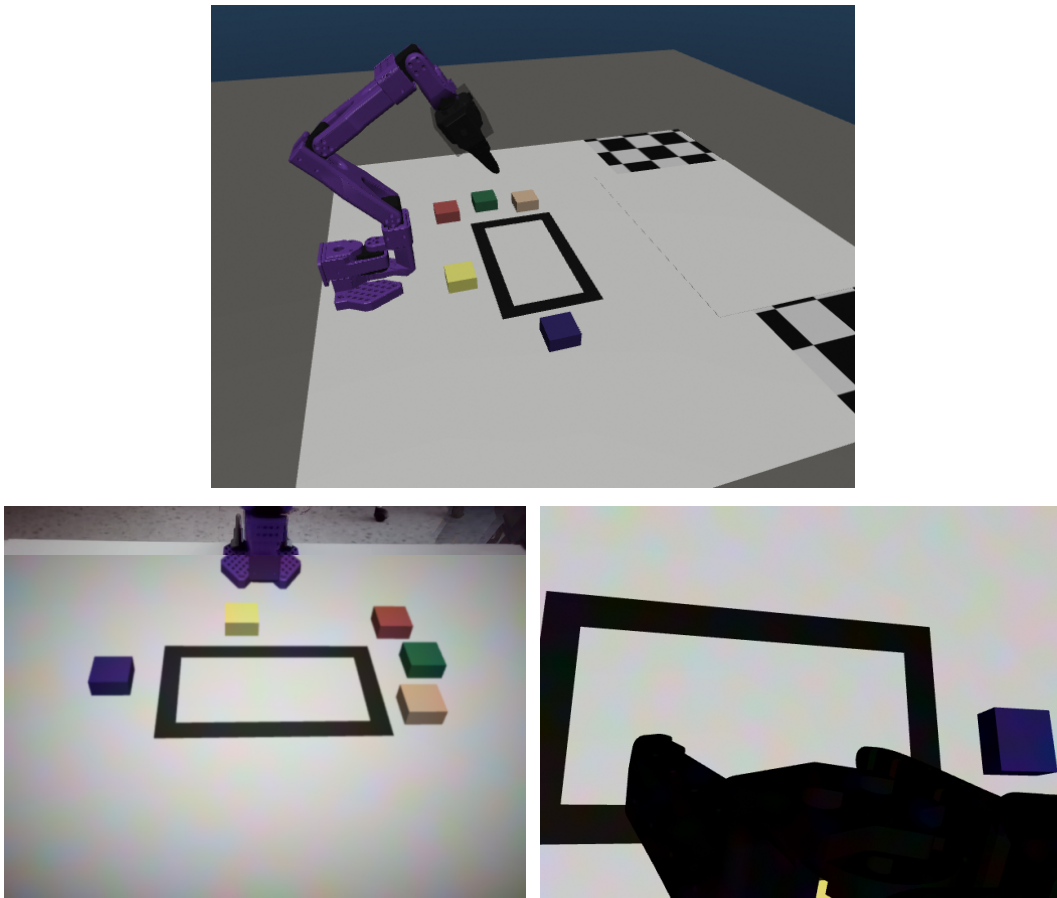


그림 3-2 MuJoCo 경기장 전체와 Side 및 Wrist 카메라 시점

시뮬레이션 환경은 SO-101 upstream MJCF를 기반으로 구축하였다. 다만 경기장, 카메라, 조명, 재질, 동역학, 시작 자세, 블록 초기 배치는 실물 환경과 맞추기 위해 별도로 보정하였다.

초기에 실물 데이터로 학습한 ACT 정책을 그대로 MuJoCo 환경에 적용했을 때 성공률은 0/5였다. 원인을 확인한 결과, 시뮬레이션에서 정책이 입력받는 영상이 실물 학습 데이터와 공간적으로 충분히 맞지 않는 문제가 있었다. 이에 따라 시뮬레이션의 정합 정도를 육안이나 추정에 의존하지 않고, 실물 데이터와 직접 비교하여 보정하는 절차를 구성하였다.

Real2Sim 보정에 사용한 실물 데이터는 다음과 같다.

표 3-5 실물과 시뮬레이션 정합에 사용한 근거

실물 근거	내용
실물 Tele-operation 데이터셋	Side/Wrist 영상, 관절 상태, 실행 action을 포함하며 카메라, 조명, 재질, 동역학, 시작 자세, 초기 배치 추정의 주요 근거로 사용
Follower 서보 캘리브레이션	my_follower_arm.json. 6개 축의 raw tick 범위를 이용해 실제 관절 가동 범위를 추정
고정 체크보드 촬영	Side/Wrist 카메라로 촬영한 영상. 카메라 intrinsic 및 배치 파라미터 추정에 사용
경기장 실측	받침대 깊이, 받침대 앞면과 테이블 사이 거리, 테이블 폭, 작업 영역 크기를 직접 측정

실물에서 측정한 값을 기준으로 시뮬레이션 파라미터를 추정하며, 결과는 보정 기록으로 저장한 뒤 MuJoCo scene XML에 반영한다.

3.4 도메인 랜덤화 설계

시뮬레이션에서 학습한 정책을 실물 환경에 적용하려면, 학습 데이터에 실제 환경에서 발생할 수 있는 변화를 충분히 반영해야 한다. 이를 위해 장면과 로봇, 관측 과정에 관련된 **179개의 파라미터를 12개의 gate**로 구분하여 관리하였다. gate는 관련 랜덤화 항목을 묶어 제어하는 단위이며, 각 축은 실제로 변화시키는 개별 파라미터를 의미한다.

gate는 파라미터의 종류보다 실물에서 해당 값이 달라지는 원인을 기준으로 구분하였다. 평가 환경에서 달라질 수 있는 요소, 사용자가 통제하기 어려운 요소, 기구의 개체차에서 발생하는 요소, 관측 및 통신 과정에서 발생하는 요소를 각각 분리하였다.

표 3-6 도메인 랜덤화의 12개 게이트

Gate	랜덤화 대상	적용 이유
A 경기장	지정 영역의 거리·치수·오프셋·회전, 테이블, 받침대 기울기, 판 마찰, 작업면 높이	지정 영역의 크기는 규정으로 정해져 있지만 실제 설치 위치와 각도에는 오차가 발생할 수 있다.
B 블록	마찰, 질량과 개체차, 접촉 물성	배부된 블록의 표면 상태와 질량 차이를 반영하고, 정책이 특정 색과 위치의 관계에 의존하는 것을 방지한다.
C 카메라 기하	마운트 자세, 카메라 회전, 화각과 주점, 렌즈 왜곡, 진동, 브래킷 처짐	삼각대와 Wrist 카메라 브래킷의 설치 오차를 반영한다.

Gate	랜덤화 대상	적용 이유
D 지연·관측	action·관측 지연, 잡음, 양자화, 프레임 드롭, 지연 스파이크, 추론 정지, 두 카메라의 비동기	서버 추론 과정에서 발생하는 네트워크 및 관측 지연을 학습 데이터에 반영한다.
G 3D 배경	뒷벽과 공간 구조, 바닥 텍스처와 마모, 정적 물체, hard negative	평가 장소의 경기장 외부 환경이 달라져도 정책이 배경에 의존하지 않도록 한다.
I 센서·ISP	잡음, 비네팅, 흐림, 노출, 화이트밸런스, 감마, 압축 아티팩트	실제 정책 입력은 카메라 촬영 이후 인코딩·디코딩 등의 처리 과정을 거친 영상이므로 해당 차이를 반영한다.
L 조명·재질	광원 세기·방향·색온도·개수, 그림자, 블록·판·로봇의 재질과 텍스처	실험실과 평가장의 조명 및 표면 상태 차이에 대한 민감도를 줄인다.
P 그리퍼	명령-개구 매핑, 파지 강도와 개구 폭, 패드 물성, 좌우 조 비대칭	그리퍼의 개체차와 서보 매핑 오차가 파지 결과에 미치는 영향을 반영한다.
R 로봇	액추에이터·관절 동역학, 링크 질량과 관성, 관절 영점과 한계, 조 유격, 서보 시정수, 조립 공차	실제 Follower Arm에서 발생하는 유격과 조립 공차 등 기구 오차를 반영한다.
S 시작 자세	6개 축의 시작 각도	실물 시연에서 매 에피소드의 시작 자세가 완전히 동일하지 않은 점을 반영한다.
T 교사	인접 블록, Pick 순서, Place 위치, 구간별 높이, 속도·정착, 정지, 실패·복구	동일한 교사 궤적만 반복하여 생성되는 것을 방지하고 행동 분포를 다양화한다.
X 외란	파지 직전 블록 밀림, 블록 넘어짐	실패 이후 복구하는 동작을 학습 데이터에 포함한다. 단, 교사가 상태 변화에 따라 재계획할 수 있는 외란만 적용한다.

마지막으로 랜덤화 파라미터에는 값의 범위뿐 아니라 변경 주기도 함께 정의하였다. 동일한 분포를 사용하더라도 에피소드마다 한 번 값을 설정하는 경우와 매 스텝 값을 변경하는 경우에는 서로 다른 데이터가 생성되기 때문이다.

따라서 도메인 랜덤화는 실측값의 분포뿐 아니라 각 물리 현상이 실제로 변화하는 시간 단위까지 함께 반영하도록 설계하였다.

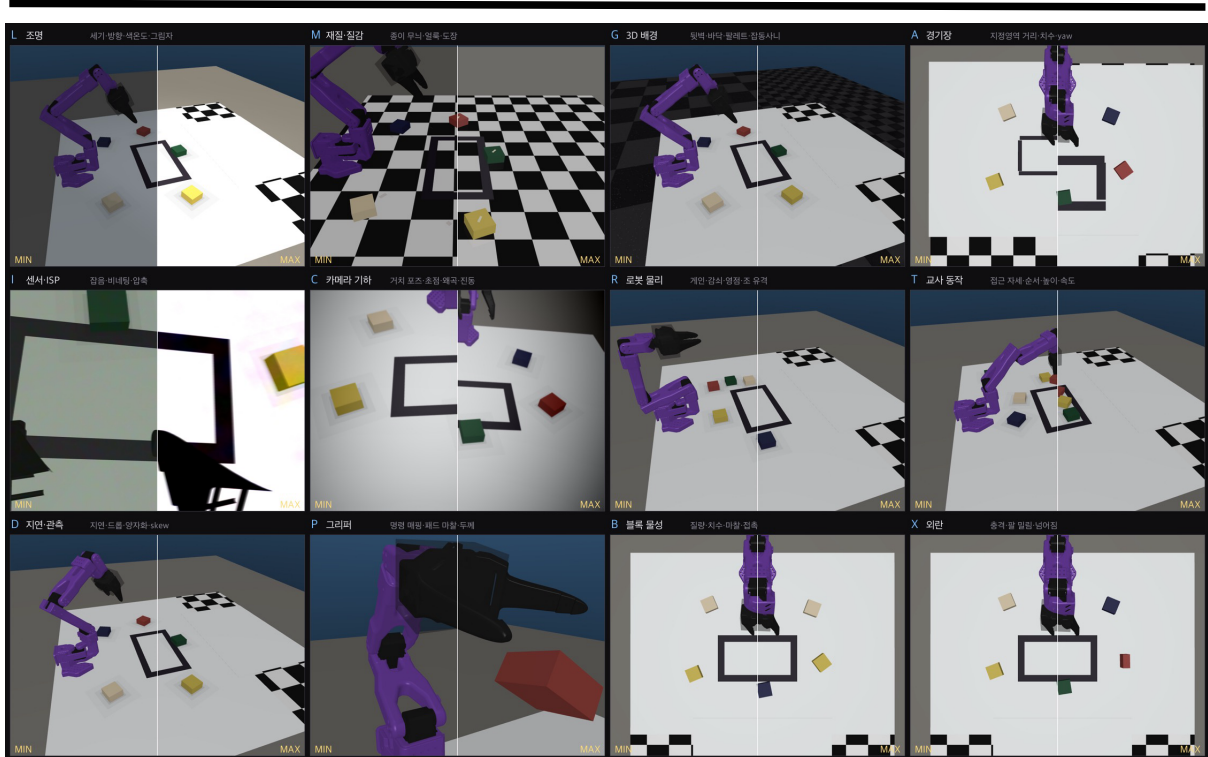


그림 3-3 12축 도메인 랜덤화 상한값, 하한값 적용 예시

3.5 IK 솔버 기반 스크립트 교사

도메인 랜덤화 환경에서 대규모 학습 데이터를 생성하기 위해 IK 기반 스크립트 교사를 구현하였다. 시뮬레이션에서는 교사가 블록의 위치와 자세를 직접 읽어 Pick & Place 및 Pick & Stack 궤적을 생성한다. 생성된 궤적은 카메라 영상, 관절 상태, action만 LeRobot Dataset 형식으로 저장하여 학습에 사용한다.

3.5.1 IK 기반 파지 자세 생성

SO-101은 5 자유도 로봇 팔이므로, end-effector의 목표 위치, 접근 방향, 그리퍼의 방위각을 동시에 정확하게 만족시키기 어렵다. 이에 위치 오차를 가장 중요하게 고려하고, 자세와 관련된 오차는 일부 허용하는 **가중 감쇠 최소자승 역기구학(weighted damped least-squares IK)**을 사용하였다.

각 목표의 가중치는 end-effector의 위치에 1.00, 접근 방향의 기울기에 0.60, 그리퍼의 방위각에 0.30을 적용하였다. 이를 통해 파지 성공에 직접적인 영향을 미치는 위치 정확도를 우선하면서도, 접근 방향과 그리퍼 회전이 지나치게 벗어나지 않도록 하였다.

블록 이동은 접근 → 하강 → 파지 → 상승 → 운반 → 정렬 → 놓기 → 후퇴 순서의 웨이포인트로 구성하였다. 긴 구간을 관절 공간에서 바로 보간할 경우 말단 경로가

뒤어져 적재된 블록과 충돌할 수 있어, 긴 수평·수직 이동 구간에는 중간 웨이포인트를 추가하였다.

3.5.2 실제 파지 가능성을 반영한 접근 자세

IK가 계산되더라도 실제 파지가 성공하는 것은 아니었다. 작업 영역을 20 mm 간격으로 나누어 4,715개 위치에서 파지를 시험한 결과, 로봇에서 멀어질수록 수직 접근의 성공률이 크게 감소하였다.

표 3-7 거리별 파지 실험과 수직 접근 성공 비율

반경	파지 성공 위치	수직 접근 성공 비율
0.10~0.20 m	406/455	86%
0.30~0.34 m	393/450	44%
0.34~0.38 m	241/515	0%
0.38~0.42 m	157/575	0%
0.42~0.50 m	182/1025	0%

따라서 IK 오차만으로 파지 가능 여부를 판단하지 않고, **위치별 실제 파지 실험**으로 성공 자세표를 생성하여 교사의 접근 자세 선택에 사용하였다. 자세표에 적절한 후보가 없으면 기존의 단계적 IK 탐색 방식으로 대체한다.

실물 시연의 접근 자세도 분석하여 거리에 따른 기울기 변화를 반영하였다. 특히 약 0.30 m 이후부터는 로봇에서 멀어질수록 기울어진 접근 자세가 필요하였다.

3.5.3 복구 동작을 포함한 데이터 생성

초기에는 성공한 시연 위주로 학습하면서, 추론 중 파지에 실패하거나 블록 위치가 달라졌을 때 정책이 다시 정상 동작으로 복귀하지 못하는 문제가 있었다. 이를 보완하기 위해 시뮬레이션 교사가 실패 상황을 감지하고 다시 행동할 수 있도록 복구 경로를 추가하였다.

파지에 실패하면 같은 위치에서 다시 접근하거나 현재 블록 위치를 기준으로 경로를 다시 계획하도록 하였다. 또한 블록이 세워져 있어 현재 그리퍼 개구 범위로 직접 잡기 어려운 경우에는 먼저 블록을 눕힌 뒤 다시 파지하도록 구성하였다. 1차 미션에서는 모든 블록에 대한 기본 동작이 끝난 뒤에도 지정 영역 밖에 남아 있는 블록이 있으면 최대 두 차례 다시 옮기도록 하였다.

3.5.4 시작 장면 다양화

복구 상황을 에피소드 중간에 추가하더라도, 모든 에피소드가 정돈된 초기 배치에서 시작하면 학습할 수 있는 상태가 제한된다. 실제 수행 중에는 이미 일부 블록이 이동했거나 서로 기대어 있는 상태에서 다음 행동을 결정해야 할 수 있으므로, 시작 상태도 다양하게 구성하였다.

일부 에피소드에서는 지정 영역 안에 2~4개의 블록이 이미 놓여 있거나 서로 기대어 있는 상태에서 시작하도록 하였다. 또 다른 경우에는 그리퍼가 다음에 집어야 할 블록의 정확한 위치가 아니라 옆으로 벗어난 위치에서 시작하도록 하였다. 이후에는 별도의 동작을 추가하지 않고 기존의 **실패 판정과 복구 로직**을 이용해 해당 상태에서 정상적인 작업으로 복구하도록 구성하였다.

3.5.5 교사 동작의 다양화

교사가 항상 동일한 궤적을 생성하지 않도록 이동 속도, 접근 방향, 놓는 위치, 이동 높이, 인접 블록, 일시 정지 등을 에피소드마다 변경하였다. 블록 이동이나 넘어짐과 같은 외란은 교사가 상태 변화를 확인하고 경로를 다시 계획할 수 있는 경우에만 적용하였다.

3.6 잔차 강화학습 설계

3.6.1 기본 정책과 잔차 행동

Residual RL은 모방학습으로 학습된 Base Policy를 고정한 상태에서, 강화학습을 통해 기존 행동에 더해지는 작은 보정값만 학습하는 구조로 설계하였다. 시각 관측과 로봇 상태가 입력되면 Base Policy가 기본 행동을 생성하고, Residual Policy는 동일한 관측을 기반으로 관절별 보정 행동을 출력한다.

최종 행동은 다음과 같이 정의된다.

$$a_t = \pi_{base}(o_t) + \Delta_{\theta}(o_t), \quad |\Delta_{\theta}(o_t)| \leq b$$

여기서 π_{base} 는 모방학습으로 사전 학습된 고정 정책, Δ_{θ} 는 강화학습으로 최적화되는 Residual Policy, b 는 관절별 잔차 허용 범위를 의미한다.

본 연구에서는 **Base Policy의 파라미터를 강화학습 과정에서 갱신하지 않았으며**, Residual Policy가 출력하는 잔차 행동에 대해서만 별도의 제한을 적용하였다. **관절별 잔차 예산**은 다음과 같이 설정하였다.

$$b = [3, 3, 3, 3, 3, 5]^{\circ}$$

잔차 크기는 단일 스칼라로 제한하지 않고, **관절별 잔차 예산**을 이용한 clipping 방식으로 제어하였다. 관절마다 가동 범위와 동작 민감도가 다르기 때문에 동일한 보정 범위를 일괄적으로 적용할 경우 일부 관절에서는 과도한 보정이 발생하거나, 반대로 필요한 보정이 지나치게 제한될 수 있다. 따라서 각 관절에 독립적인 잔차 허용 범위를 부여하였다.

또한 학습과 실제 배포 과정에서 동일한 잔차 스케일이 적용되도록 관련 계산을 공통 모듈로 통합하였다.

3.6.2 보상 함수 설계

전체 보상은 크게 매 제어 시점마다 계산되는 dense reward와 특정 조건이 처음 충족될 때 한 번만 지급되는 event reward로 구성하였다.

Dense 및 event reward를 사용하는 경우 전체 보상은 다음과 같다.

$$r_t = r_{progress} + r_{grasp} + r_{place} + r_{completion} + r_{speed} - r_{residual}$$

(1) Dense reward

Dense reward는 작업 수행 과정의 방향을 안내하는 potential progress와 Residual Policy의 과도한 보정을 억제하는 residual penalty로 구성하였다.

가. Potential progress

작업 진행도는 현재 거리 자체가 아니라 상태 potential의 변화량을 이용하여 계산하였다.

$$r_{progress} = 0.25 [\gamma \Phi(s_{t+1}) - \Phi(s_t)]$$

Potential $\Phi(s)$ 는 이미 배치가 완료된 블록 수와 현재 대상 블록의 진행도를 합한 값으로 정의하였다. 현재 대상 블록의 진행도는 파지 전과 파지 후가 하나의 연속적인 값으로 이어지도록 구성하였다.

1. 파지 전에는 그립퍼와 블록 사이의 접근 거리와 중심 정렬 정도를 반영하였다.
2. 파지 후에는 블록과 목표 슬롯 사이의 평면상 거리를 반영하였다.
3. 파지 시점을 기준으로 진행도가 자연스럽게 연결되도록 phase 범위를 구성하였다.

접근 거리에는 tanh 기반의 coarse 항을 사용하였다. 다만 목표 지점에 가까워질수록 해당 항의 변화량이 작아지는 반면, 실제 파지 성공 여부는 마지막 수 mm 수준의 정렬 차이에 의해 결정될 수 있다. 이를 보완하기 위해 Gaussian 형태의 grasp_center_quality를 fine 항으로 추가하였으며, scale은 15 mm로 설정하였다.

fine 항의 가중치는 0.05를 사용하였다. 이전 실험에서는 0.25를 적용하였으나 **중심 정렬 항이 전체 진행도를 과도하게 지배하는** 경향이 관찰되어 현재 값으로 축소하였다.

또한 potential-based shaping이 작업의 최종 목표 자체를 변경하지 않고 중간 행동의 방향만 제공하도록, 에피소드 종료 시 **terminal potential을 제거**하는 보정을 적용하였다. 이를 통해 dense progress 항은 빠른 접근과 운반을 유도하되, 최종적인 성공 여부와 성능 평가는 event reward가 결정하도록 구성하였다.

나. Residual penalty

Residual Policy가 Base Policy의 행동을 필요 이상으로 크게 변경하지 않도록 다음 두 항에 대해 감점을 적용하였다.

1. 잔차 행동 자체의 크기
2. 잔차의 시간적 변화량에 대한 규제

잔차 정책이 기본 행동을 과도하게 수정하거나 제어 주기마다 급격하게 변하는 것을 방지하기 위해 두 가지 감점 항을 적용하였다.

먼저, 잔차 행동의 크기에 대한 감점을 적용하여 보정값이 필요한 범위 이상으로 커지는 것을 억제하였다. 또한 연속된 제어 주기 사이의 잔차 변화량에 감점을 적용하여 관절 명령이 갑작스럽게 변하지 않도록 하였다.

각 관절은 허용되는 최대 보정 범위가 서로 다르므로, 잔차 행동과 그 변화량을 관절별 보정 한도(b_i)로 정규화하였다. 이를 통해 관절마다 허용 범위가 다르더라도 동일한 기준으로 보정의 크기와 변화량을 평가할 수 있다.

잔차 행동의 크기에 대한 감점 계수는 0.002, 연속된 제어 주기 사이의 잔차 변화량에 대한 감점 계수는 0.001로 설정하였다.

또한 residual penalty는 에피소드의 step budget에 따라 스케일링하였다. 이를 통해 잔차가 에피소드 전체 구간에서 최대 크기로 출력되더라도 누적 감점이 약 -0.8 수준으로 유지되도록 하였다.

해당 스케일링을 적용하지 않을 경우 5개 블록을 수행하는 긴 에피소드에서 누적 감점이 약 -10.8까지 증가할 수 있다. 이 경우 작업 성공을 위해 잔차를 사용하는 것보다 잔차를 사용하지 않는 정책이 더 높은 보상을 받을 수 있으므로, Residual Policy가 사실상 $\Delta_t = 0$ 에 가까운 행동만 출력하는 방향으로 수렴할 가능성이 있다. 이를 방지하기 위해 **에피소드 길이에 따른 감점 보정**을 적용하였다.

(2) Event reward

Event reward는 작업 수행 과정에서 의미 있는 조건이 처음 충족되었을 때만 지급한다. 본 연구에서는 **grasp, place, completion, speed**의 네 가지 event reward를 사용하였다.

가. Grasp event

파지 보상은 블록당 한 번만 지급한다. 파지 여부는 다음 두 조건을 모두 만족할 때 판정한다.

1. 블록이 그리퍼 중심으로부터 5 cm 이내에 위치
2. 블록의 최저점이 바닥으로부터 1.5 cm 이상 상승

파지에 성공하면 최소 0.125의 보상을 지급하며, 그리퍼와 블록 중심의 정렬 정확도가 높을수록 최대 0.5까지 보상이 증가한다.

따라서 단순히 블록을 들어 올리는 행동에도 일부 보상을 제공하면서, 블록 중심을 정확하게 파지한 경우 더 높은 보상을 부여하도록 구성하였다.

나. Place event

배치 보상은 블록당 한 번만 지급한다. 공식적으로 배치에 성공하면 1.0의 기본 보상을 지급하고, 슬롯 중심과의 정렬 정확도에 따라 최대 0.5의 추가 보상을 부여한다. 이에 따라 블록당 배치 보상은 1.0에서 1.5 사이의 값을 갖는다.

성공 판정과 정밀도 보상을 함께 사용한 이유는, 정밀도 항만 독립적으로 사용할 경우 공식적으로 실패한 배치를 반복적으로 미세 조정하는 행동이 정상적으로 성공한 배치보다 높은 보상을 획득할 가능성이 있기 때문이다.

다. Completion 및 Speed event

모든 대상 블록을 성공적으로 배치한 경우 2.0의 completion reward를 추가로 지급하였다.

또한 5개 블록 전체를 수행하는 태스크에서는 전체 작업에 성공한 경우에만 남은 시간에 따라 최대 2.0의 speed reward를 부여하였다.

시간 보너스를 전체 완료 조건에 종속시킨 이유는 실패한 정책이 에피소드를 조기에 종료하는 방식으로 보상을 획득하는 것을 방지하기 위함이다. 따라서 속도는 작업 성공이 확인된 정책 사이에서만 추가적인 평가 요소로 작용한다.

보상 구성을 정리하면 다음과 같다.

표 3-8 잔차 강화학습의 보상 구성

구분	보상 항목	크기	지급 빈도
Dense	Potential progress	$0.25 \times (\gamma\Phi' - \Phi)$	매 tick
Dense	Residual penalty	에피소드 합 약 -0.8	매 tick
Event	Centered grasp	0.125–0.5	블록당 1회
Event	Valid place	1.0–1.5	블록당 1회
Event	Completion	2.0	에피소드당 1회
Event	Speed	0–2.0	에피소드당 1회

5개 블록 기준 event reward의 이론적 최대값은 약 14.0이다. 파지 보상 최대 2.5, 배치 보상 최대 7.5, completion reward 2.0, speed reward 최대 2.0으로 구성된다.

3.6.3 학습 구조

Residual Policy는 replay buffer와 n-step return을 사용하는 off-policy 방식으로 학습하였다. 학습기는 TD3/REDQ 계열의 twin critic과 비동기 learner 구조로 구성하였으며, 환경에서 경험을 수집하는 과정과 네트워크 파라미터를 갱신하는 과정을 분리하였다.

주요 학습 구성은 다음과 같다.

1. Replay buffer를 이용한 경험 재사용
2. n-step return을 이용한 장기 보상 반영
3. Twin critic을 이용한 **value overestimation** 완화
4. 환경 rollout과 learner update의 비동기 실행
5. 시뮬레이션 데이터 수집과 정책 업데이트의 병렬화

기존 모방학습 데이터도 초기 replay buffer에 포함하였다. 시연 데이터에서는 Base Policy의 행동을 기준 행동으로 사용하고 Residual action을 0으로 재표기하였다.

$$\Delta_t = 0$$

이를 통해 학습 초기에는 Residual Policy가 Base Policy의 행동을 크게 변경하지 않도록 유도하고, 이후 환경과의 상호작용을 통해 실제 작업 성공에 필요한 보정 행동만 점진적으로 학습하도록 하였다.

잔차 예산의 적용 목적

제한된 잔차는 **파지 중심 정렬**, 손목 각도, 그리퍼 폐쇄 시점, 목표 슬롯 접근 오차를 보정하기 위한 것이다. 앞서 정의한 관절별 예산 b 를 공통 모듈에서 적용하며, 기본 정책의 전체 동작을 대체하기보다 국소 오차를 수정하는 것을 목표로 한다. 잔차 범위 제한만으로 작업 성공률이 보장되지는 않으므로, 성능 개선 여부는 별도의 비교 평가로 판단하였다.

3.6.4 보상 구현 검증 및 오류 분석

Residual RL 학습 과정에서 dense reward의 일부가 의도대로 활성화되지 않는 문제가 확인되었다. 보상 shaping은 현재 정책이 어떤 블록을 목표로 하고 있는지를 기준으로 진행도를 계산하도록 구현되어 있었으나, 스크립트 기반 교사는 목표 블록 정보를 제공하는 반면 ACT Base Policy는 해당 정보를 반환하지 않았다.

이로 인해 초기 6회의 Residual RL 학습에서는 기준 블록을 필요로 하는 **dense reward** **항이 비활성화**된 상태로 실행되었다.

이를 해결하기 위해 **현재 시점에서 가장 가까운 미배치 블록**을 목표 블록으로 추론하는 방식으로 수정하였다. 해당 추론 방식이 실제 Base Policy의 행동과 일치하는지도 별도로 검증하였다. 잔차 행동을 0으로 설정한 상태에서 Base Policy만 실행하고, 이후 실제로 처음 파지되는 블록을 사후적으로 정답 블록으로 정의한 뒤, 각 tick에서 추론된 목표 블록과 비교하였다.

표 3-9 추론한 목표 블록과 실제 파지 블록의 일치율

구간	일치율
파지 직전 1초	0.978
접근 구간 전체	0.706

파지 직전 1초 구간에서는 97.8%의 높은 일치율을 보였다. 반면 접근 구간 전체의 일치율은 70.6%로 낮았는데, 대부분의 불일치는 그리퍼가 모든 블록으로부터 멀리 떨어진 초기 이동 구간에서 발생하였다.

이 구간에서는 distance-based potential이 이미 포화되어 있어 reward gradient의 크기가 매우 작다. 예를 들어 블록과의 거리가 5 cm일 때의 gradient 크기와 비교하면 30 cm 부근에서는 약 78배 작아, 잘못된 목표 블록을 선택하더라도 해당 구간의 영향이 상대적으로 작을 가능성을 시사하였다.

다만 이 분석만으로 학습 결과에 미치는 영향을 정량적으로 확정할 수는 없다.

수정 이후 동일한 조건에서 학습을 다시 수행하였으나, 기존과 유사한 성능 저하가 계속 관찰되었다. 따라서 dense reward 일부가 비활성화된 문제는 실제 구현 오류였지만, 해당 오류만으로 학습 실패를 설명할 수는 없었다.

이 과정은 초기에는 명확한 실패 원인으로 판단했던 요소가 후속 실험을 통해 해당 요소만으로 전체 성능 저하를 설명할 수 없음을 확인한 사례이다. 따라서 이후 학습 실패 분석에서는 단일 구현 오류에 원인을 귀속하지 않고, reward scale, residual budget, exploration 특성, Base Policy의 상태 분포 등 다른 요인들을 함께 검토하도록 하였다.

3.7 Human-in-the-Loop 교정 데이터 수집 설계

기존 모방학습 정책은 사람이 직접 수행한 성공 시연을 기반으로 학습하므로, **학습 데이터에 포함되지 않은 상태**에 진입했을 때 오류를 복구하기 어렵다. 이를 보완하기 위해 Human-in-the-Loop(HIL) 기반의 교정 데이터 수집 방식을 적용하였다.

HIL에서는 **정책이 우선적으로 로봇을 제어**하고, 실패가 예상되는 구간에서만 사람이 개입하여 행동을 교정한다. 이렇게 수집된 교정 구간을 기존 데이터에 추가하여 재학습함으로써, 정책이 기존 시연 데이터에 없던 실패 직전 상태와 복구 행동을 학습할 수 있도록 하였다.

전체 과정은 다음과 같이 구성하였다.

Policy 실행 → 실패 가능 상태 진입 → 사용자 개입 → 행동 교정 → Policy 제어 복귀 → 교정 데이터 추가 및 재학습

HIL 구현 시에는 다음 세 가지 요소를 중심으로 설계하였다.

(1) 정책과 사용자 간 제어권 전환

1. Policy 제어 중 사용자가 개입하면 기존 action queue를 즉시 비운다.
2. 사용자 개입이 종료되면 현재 로봇 상태를 기준으로 새로운 policy action을 요청한다.
3. 이를 통해 개입 이전 상태에서 생성된 action이 이후 다시 실행되는 문제를 방지하였다.

(2) 제어 전환 구간의 연속성 유지

1. **Policy 제어에서 Human 제어**로 바로 전환하면 Leader Arm과 Follower Arm의 자세 차이로 인해 궤적이 불연속적으로 기록될 수 있다.
2. 이를 방지하기 위해 정렬 대기 단계를 두고, Leader Arm이 현재 Follower Arm 자세에 충분히 가까워진 이후 Human 제어로 전환하였다.
3. 이를 통해 Policy 구간과 Human 구간이 자연스럽게 이어지도록 하였다.

(3) 실제 실행 action 기준 데이터 기록

1. 데이터에는 정책이 생성한 action이 아니라 **실제 Follower Arm에 전달된 action**을 저장하였다.
2. 따라서 Policy 구간과 Human 개입 구간을 하나의 연속된 trajectory로 구성할 수 있으며, 실제 로봇이 수행한 행동을 기준으로 학습 데이터를 구축할 수 있다.

수집된 HIL 데이터는 기존 성공 시연을 대체하는 것이 아니라, **Base Policy가 취약한 상태와 복구 행동을 선택적으로 추가하는 보완 데이터**로 활용하였다. 이를 통해 전체 데이터를 다시 수집하지 않고도 실패가 자주 발생하는 구간을 중심으로 학습 데이터를 확장할 수 있도록 하였다.

4 연구 결과 분석 및 평가

본 장에서는 실물 로봇의 작업 수행 성능을 높이기 위해 진행한 데이터 개선과 정책 비교 실험을 정리한다. 먼저 시뮬레이션 데이터의 배치 분포, 도메인 랜덤화 및 초기 상태를 단계적으로 개선한 과정을 설명한다. 이어 실물 단독, 시뮬레이션 단독, 실물·시뮬레이션 혼합 데이터의 학습 효과와 ACT·SmolVLA의 성능을 비교하여 실물 배포에 사용할 데이터 구성과 정책을 선정한다. 또한 파지 정확도 개선을 위해 시도한 잔차 강화학습의 결과와 한계를 분석한다.

중간 실험과 최종 평가의 연결

중간보고서에서는 실물 Place 시연 100개로 학습한 SmolVLA가 단순 배치에서 5개 블록을 옮기는 사례를 기록하였다. 수행 시간은 약 4분으로, 최종 Place 제한시간 3분 내 성능과 직접 동일하게 취급할 수 없다. 어려운 위치에서 파지 실패가 증가하여 카메라 구성과 정책을 함께 재검토하였다.

ACT와 Stack 시연 50개를 사용한 카메라 비교에서는 Top+Wrist에서 헛손질, Top+Side에서 초기 정지 현상이 관찰되었다. Side+Wrist는 파지와 이동이 상대적으로 원활했으나 놓기 단계에서 간헐적 실패가 남았다. 중간 단계에서는 배경 노이즈와 시뮬레이션 정합 부담을 고려해 낮은 각도의 Top+Wrist를 절충안으로 정하였다. 최종 카메라 배치는 3.1.2절에 제시하였다.

표 4-1 중간 단계의 정책 및 카메라 변경과 지연 시간

측정 항목	중간 기존 조건	중간 변경 조건
정책과 카메라	SmolVLA와 3대	ACT와 2대
추론 시간	241.55 ms	약 11 ms
역직렬화 시간	약 100 ms	약 0.6 ms

이 비교는 정책 모델과 카메라 수를 동시에 바꾼 결과다. 따라서 지연 감소를 카메라 수 또는 모델 변경 하나의 효과로 분리하여 해석하지 않는다. 최종 정책 비교는 4.3절에서 동일한 혼합 데이터 조건의 실물 결과로 제시한다.

4.1 시뮬레이션 데이터 개선 과정

시뮬레이션 데이터 생성기는 초기 설계를 고정하여 사용한 것이 아니라, **생성된 데이터로 학습한 정책을 다시 시뮬레이션 혹은 실물에서 평가하고, 관찰된 실패 원인을 데이터 분포와 생성 조건에 반영하는 반복 과정**을 통해 개선하였다.

특히 초기 블록 배치 분포, 도메인 랜덤화 범위, 데이터 채택 조건, 초기 장면 구성 등을 단계적으로 수정하였다.

4.1.1 초기 블록 배치 분포 개선

초기 블록 배치 방식은 slot → sector → free 순서로 변경하였다.

표 4-2 블록 초기 배치 분포의 개선

단계	배치 방식	특징	변경 이유
초기	slot	실물 데이터 수집에 사용 한 5개 위치 주변에 배치	실물 데이터의 초기 위치 분포 재 현
08-25	sector	경기장을 5개 구역으로 나 누어 각 구역에 하나씩 배 치	특정 위치 패턴에 대한 과적합 완 화
08-26 이후	free	경기장 전체에서 자유롭게 배치	평가와 데이터 생성의 위치 분포 통일 및 위치 다양성 확대

slot 방식으로 생성된 데이터로 학습한 정책에서는 **모든 블록이 한쪽에 배치되어 있어도 기존 학습 데이터에서 블록이 자주 등장하던 위치를 탐색하는 행동**이 관찰되었다. 이는 정책이 실제 블록의 위치보다 반복적으로 등장한 배치 패턴에 의존하여 학습했을 가능성을 보여준다.

이에 위치 분포를 넓히기 위해 sector 방식을 도입하였고, 이후에는 경기장 전체에서 블록 위치를 자유롭게 샘플링하는 free 방식으로 변경하였다. 실제 대회에서도 블록이 정해진 위치가 아닌 **임의의 위치에 배치되므로**, 학습 데이터 역시 특정 위치나 구역에 의존하지 않도록 구성할 필요가 있었다.

또한 교사 정책의 성능 평가 역시 free 배치를 기준으로 수행하도록 통일하여, **학습 데이터 생성 조건과 평가 조건이 동일한 위치 분포를 따르도록 구성하였다**. 이를 통해 다양한 초기 배치에 대한 정책의 일반화 성능을 높이고자 하였다.

4.1.2 도메인 랜덤화 확장

도메인 랜덤화는 한 번에 전체 조건을 적용하지 않고, 학습 방식과 데이터 목적에 따라 단계적으로 확대하였다.

초기에는 고정된 ACT 정책을 사용하고 있어 랜덤화가 오히려 성능을 저하시켰으며, 이후 센서·재질부터 물리·지연 요소까지 순차적으로 적용 범위를 넓혔다. 실물 데이터와 혼합 학습하던 단계에서는 **로봇의 시작 자세와 센서·ISP 관련 랜덤화만 적용**하였다.

이후 일반화 성능 확보를 위해 12개 게이트 전체를 적용하였다. 다만 전체 랜덤화 적용 직후 채택 수율이 크게 감소하여, 각 랜덤화 요소의 영향을 다시 확인하고 일부 조건을 조정된 뒤 과제별로 적용 범위를 재설정하였다.

즉, 도메인 랜덤화는 가능한 요소를 모두 적용하는 방식이 아니라, 학습 목적과 데이터 수율을 함께 고려하여 단계적으로 확장하였다.

4.1.3 다양한 초기 상태를 포함한 데이터 생성

초기 시뮬레이션 데이터는 대부분 지정 영역이 비어 있는 정돈된 상태에서 시작하였다. 실물 평가 과정에서 지정 영역에 놓여 있던 블록을 일부 정리한 뒤 정책 성능이 크게 향상되는 현상이 관찰되었으며, 이를 통해 정책이 깨끗한 지정 영역에 편향되어 학습되었을 가능성을 확인하였다.

이에 실제 수행 중 발생할 수 있는 상태를 학습하기 위해 두 종류의 시작 장면을 추가하였다.

1. **불규칙 시작:** 지정 영역 내부에 2~4개의 블록이 이미 놓여 있거나 일부 블록이 겹쳐 있는 상태에서 시작하도록 하여, 수행 도중 영역이 어질러진 상황을 학습 데이터에 포함하였다.
2. **실패 복구 시작:** 블록을 처음부터 정상적으로 접근하는 대신, 그리퍼가 목표 블록을 빗나간 채 아래로 내려간 상태에서 시작하도록 하였다. 이후 로봇이 다시 상승한 뒤 블록에 재접근하여 파지를 시도하는 궤적을 생성함으로써 초기 파지 실패 이후의 복구 행동을 학습 데이터에 포함하였다.

4.1.4 데이터 생성 비용과 재현성 개선

전체 도메인 랜덤화를 적용한 1,000 episode 생성에는 약 8.5시간이 소요되었다.

물리 시뮬레이션과 이미지 처리 속도를 개선한 이후에는 AV1 영상 인코딩 시간이 rollout보다 길어져, 주요 병목이 시뮬레이션 연산에서 데이터 저장 단계로 이동하였다.

이후 데이터 규모 확대, 도메인 랜덤화, 불규칙 초기 배치를 순차적으로 적용하여 다양한 상태와 환경 변화를 포함하도록 데이터 구성을 확장하였다.

4.1.5 최종 시뮬레이션 데이터 구성

앞선 데이터 생성기 개선 사항을 종합하여 최종적으로 sim_place_1000_dirty300_0908 데이터셋을 구축하였다. 해당 데이터셋에는 다음 세 가지 조건을 반영하였다.

1. **전체 도메인 랜덤화 적용:** 12개 랜덤화 게이트를 적용하여 다양한 시각·물리 환경을 포함하였다.

2. 다양한 시작 상태 추가: 일반 시작 700 episode와 불규칙·실패 복구 시작 300 episode를 목표로 구성하였다.

3. 자유 배치 적용: 블록을 경기장 전체에 무작위로 배치하여 실제 대회의 임의 배치 조건을 반영하였다.

최종적으로 **1,000 episode**의 시뮬레이션 데이터를 확보하였다. 학습 시 실물 데이터 170 episode를 결합하여 place_dirty300sim_real_0908_1170 데이터셋을 구성하였다.

4.2 학습 데이터 구성 비교

4.1절에서 확정한 시뮬레이션 데이터를 기반으로, Real only / Sim only / Sim + Real 세 가지 데이터 조합으로 각각 ACT를 학습하고 실물 로봇에서 rollout 성능을 비교하였다.

학습 데이터에 따른 성능 (실물 로봇 rollout 기준)

정책: ACT / Task: Pick and Place

표 4-3 학습 데이터별 실물 평가 결과

학습 데이터	평균 완료	관찰된 행동 특징
Real only	2.2/5개	익숙한 배치에서는 안정적이거나 낯선 배치에서 수행 불안정
Sim only	2.3/5개	낯선 배치에서도 손목 회전·실패 복구 능력 향상; 다만 관절 유격 등 Sim2Real 격차로 실물 파지 실패 발생
Sim + Real	3.4/5개	다양한 상황 대응 및 Sim2Real 격차 완화; Real/Sim only보다 안정적이거나, 초기 파지 실패가 전체 실패로 이어짐

1. 제한시간 180초 이내 성공 개수 기준

2. 각 조건당 20회 수행 / 행동 기록: 파지 실패, 낙하, 재시도 등

Real only의 낯선 배치 불안정은 실물 데이터만으로 확보 가능한 배치 다양성의 한계로, Sim only의 파지 실패는 시뮬레이션과 실물 로봇 사이의 물리적 차이(관절 유격 등)로 인한 sim-to-real 격차로 추정된다. Sim 데이터가 제공하는 대응력과 Real 데이터가 좁히는 sim-to-real 격차가 함께 작용해 Sim + Real 조합이 세 조합 중 가장 우수한 것으로 확인되었으며, 이에 따라 이후 정책 비교는 Sim + Real 데이터로 통일하여 진행하였다.

다만 동일한 초기 배치를 조건 간 짝지어 사용했는지와 학습 시드 반복 여부는 제공 기록에 명시되지 않아, 이 결과는 해당 평가 조건에서의 관찰로 해석한다.

4.3 ACT와 SmoIVLA 정책 비교

Sim + Real 데이터로 ACT와 SmoIVLA를 각각 학습하여 실물 로봇에서 Roll-out하였다.

정책 모델에 따른 성능

학습 데이터: Sim + Real / Task: Pick and Place

표 4-4 정책 모델별 실물 평가 결과

정책 모델	평균 완료	관찰된 행동 특징
ACT	3.4/5개	목표 큐브 접근은 안정적이거나 파지 정확도 부족
SmoIVLA	1.6/5개	목표 접근이 불안정해 지나치거나 빈 위치로 이동하는 경우 발생

1. 제한시간 180초 이내 성공 개수 기준
2. 각 조건당 20회 수행 / 행동 기록: 파지 실패, 낙하, 재시도 등

ACT는 큐브까지의 접근 경로 자체는 안정적으로 유지하면서 파지 정확도에서만 약점을 보인 반면, SmoIVLA는 접근 단계에서부터 목표를 지나치거나 엉뚱한 위치로 향하는 등 더 근본적인 불안정성을 드러냈다. 이 결과를 근거로 이후 학습은 ACT를 base 정책으로 채택하여 진행하였다.

4.4 잔차 강화학습 실험 결과

앞선 실물 정책 비교에서 ACT가 SmoIVLA보다 높은 평균 완료 수를 보여, 이후 실험에서는 ACT를 기본 정책으로 채택하였다. 다만 실물·시뮬레이션 혼합 데이터로 학습한 ACT에서도 파지 정확도 부족이 반복적으로 관찰되었다. 이를 개선하기 위해 잔차 강화학습(Residual Reinforcement Learning)을 적용하였으며, 기본 정책의 가중치를 고정한 상태에서 잔차 정책이 기본 행동에 작은 보정값을 더하도록 구성하였다.

실험은 MuJoCo 시뮬레이션에서 SO-101 로봇이 **5개의 블록을 목표 영역으로 옮기는 작업**을 대상으로 수행하였다. 기본 정책으로는 시뮬레이션 데이터로 학습한 **ACT 0827**과 실물 배포를 고려하여 실물·시뮬레이션 혼합 데이터로 학습한 **ACT 0905**를 사용하였다.

4.4.1 평가 방법

잔차 정책의 효과는 **동일한 평가 시드**에서 기본 정책 단독 실행과 잔차 정책 결합 실행을 비교하여 측정하였다. 주요 평가 지표는 다음과 같다.

1. **평균 배치 수**: 에피소드당 목표 영역에 배치한 블록 수(최대 5개)
2. **전체 성공률**: 5개 블록을 모두 배치한 에피소드의 비율
3. **성능 변화량(Δ)**: 잔차 정책 결합 후 평균 배치 수에서 기본 정책의 평균 배치 수를 뺀 값

Δ 가 양수이면 성능 개선, 음수이면 성능 저하를 의미한다. 초기에는 조건별 10에피소드로 평가하였으며, 후속 규모 확대 실험에서는 평균 추정의 불확실성을 줄이기 위해 **50에피소드**로 확대하였다.

4.4.2 보상 함수 점검 및 수정

초기 실험에서는 학습량, 환경 무작위화 범위, 가치함수 구조 및 시각 입력 사용 여부를 변경하였다. 이 과정에서 **ACT를 기본 정책으로 사용할 때 일부 보상이 계산되지 않는 구현 결함**을 발견하였다.

기존 보상 함수는 기본 정책이 제공하는 목표 블록 정보를 사용하였다. 그러나 ACT는 목표 블록을 명시적으로 출력하지 않아, 다음 두 보상이 비활성화되어 있었다.

1. 블록에 접근하거나 목표 영역으로 운반하는 과정의 **진행도 보상**
2. 블록을 들어 올렸을 때 부여하는 **파지 보너스**

이를 해결하기 위해 들고 있는 미완료 블록을 우선 선택하고, 해당 블록이 없으면 그리퍼와 가장 가까운 미배치 블록을 보상 계산 대상으로 사용하였다. **잔차 크기와 변화량**에 대한 감점도 에피소드 시간 예산에 맞춰 조정하였다.

수정 후 보상이 정상적으로 발생하는 것은 확인하였다. 그러나 ACT 0827의 평균 배치 수는 기본 정책 **3.3개**에서 **잔차 결합 후 1.9개로 감소**하였다. 따라서 **보상 구현 결함의 수정만으로 성능 개선을 달성하지는 못하였다**.

4.4.3 탐색 범위 및 반복 학습 실험

잔차 행동의 탐색 범위가 학습에 미치는 영향을 확인하기 위해 탐색 노이즈 계수를 변경하였다. 이 계수는 잔차 허용 범위에 대한 가우시안 노이즈의 표준편차 비율이며, 평가 시에는 노이즈를 제거하였다.

ACT 0827을 대상으로 **100,000 step** 학습 후 10에피소드를 평가한 결과는 다음과 같다.

1. **탐색 노이즈 0.2**: 평균 배치 수 $3.3 \rightarrow 1.9$ 개, $\Delta = -1.4$ 개

2. 탐색 노이즈 0.5: 평균 배치 수 3.0 $\rightarrow$ 2.6개, $\Delta = -0.4$ 개

3. 탐색 노이즈 1.0: 평균 배치 수 3.2 $\rightarrow$ 2.6개, $\Delta = -0.6$ 개

탐색 범위를 확대한 조건에서 성능 저하 폭이 작게 관찰되었지만, 모든 조건에서 기본 정책보다 낮은 성능을 보였다.

학습 과정의 무작위성에 따른 차이를 확인하기 위해 탐색 노이즈 0.2 조건에서 학습 시드를 변경하여 세 차례 추가 실험을 수행하였다. 기존 실행을 포함한 네 실행의 Δ 는 $-1.4, -0.9, -1.7, -1.3$ 개였다. 네 실행 모두 성능이 감소했으나, 탐색 노이즈 0.5와 1.0 조건은 각각 한 차례만 학습하였으므로 조건 간 차이를 확정적인 효과로 해석하지 않았다.

4.4.4 잔차 출력 크기 및 평가 신뢰성 점검

학습된 잔차의 크기가 과도한지 확인하기 위해, 추가 학습 없이 잔차 출력에 0, 0.25, 0.5, 1.0의 계수를 곱하여 평가하였다. 계수 0은 기본 정책만 실행하는 조건이며, 계수 1은 학습된 잔차를 그대로 적용하는 조건이다.

일부 체크포인트에서는 잔차를 축소했을 때 기본 정책보다 높은 평균 배치 수가 관찰되었다. 그러나 평가 표본이 10에피소드로 제한되어 일관된 개선을 확인하기 어려웠다.

동일 체크포인트와 평가 시드를 사용한 재평가에서도 결과 차이가 나타났다. 이는 표본 수 부족만으로 설명할 수 없으며, 실행 환경·난수 처리·정책 실행 경로·수치적 비결정성에 대한 추가 점검이 필요한 부분이다. 이에 후속 실험에서는 평가를 50에피소드로 확대하되, 실행 재현성 문제는 별도의 검증 과제로 구분하였다.

4.4.5 학습 규모 확대 및 행동 반복 실험

장기간 학습의 효과를 확인하기 위해 경험량을 약 100만 제어 주기로 확대하고, 리플레이 버퍼 용량을 50,000개에서 200,000개 전으로 늘렸다.

또한 기본 정책은 30Hz로 동작하게 유지하면서, 잔차 보정값만 3개 제어 주기 동안 유지하는 행동 반복(action repeat, $k=3$)을 적용하였다. 이 조건에서 잔차 정책은 약 100ms마다 새로운 보정값을 결정한다.

완료된 $k=3$ 실험은 333,000회의 잔차 결정으로 구성되었다. 각 결정이 3개 제어 주기 동안 유지되므로, 종료에 따른 단축이 없을 경우 총 999,000회의 제어 주기에 해당한다.

(1) 50에피소드 평가 결과

- ACT 0827

- 평균 배치 수: 2.76 $\rightarrow$ 2.06개

- 성능 변화량: $\Delta = -0.70$ 개, 표준오차 약 0.27개
- 전체 성공률: 14% → 6%

- **ACT 0905**

- 평균 배치 수: 0.84 → 0.66개
- 성능 변화량: $\Delta = -0.18$ 개, 표준오차 약 0.23개
- 전체 성공률: 2% → 0%

ACT 0827에서는 해당 평가에서 성능 저하가 관찰되었다. ACT 0905는 변화량에 비해 추정의 불확실성이 커 개선 또는 저하를 명확히 판단하기 어려웠다. **두 모델 모두 기본 정책을 안정적으로 능가하는 결과는 확인하지 못하였다.**

행동 반복은 잔차 결정 주기뿐 아니라 물리 시간에 대한 할인율, 업데이트 횟수 및 버퍼가 담는 경험의 시간 범위도 함께 변경한다. 따라서 결과를 행동 반복 하나의 효과로 해석하는 데에는 한계가 있다. $k=1$ 대조 실험과 약 200만 제어 주기의 추가 실험도 시작하였으나, **서버 접속 장애** 이후 최종 결과를 확인하지 못한 실행은 본 분석에서 제외하였다.

4.4.6 종합 결론 및 향후 과제

본 실험 범위에서는 잔차 강화학습을 통한 안정적인 성능 개선을 확인하지 못하였다. 다만 보상 계산 결함을 발견·수정하고, 탐색 범위·잔차 크기·학습 규모·결정 주기에 따른 결과를 수집하였다.

잔차 정책이 0을 출력하면 기본 정책과 동일하게 행동할 수 있지만, 학습 과정에서 기본 정책 이상의 성능이 보장되는 것은 아니다. 잔차 정책은 실제 성공률이 아닌 가치함수의 추정치를 기반으로 갱신되므로, 가치 추정에 오차가 있거나 보상이 최종 평가 지표와 어긋나면 불리한 보정값을 학습할 수 있다.

현재 결과만으로 성능 저하의 원인을 하나로 확정하기는 어렵다. 향후에는 다음 사항을 우선 검증할 필요가 있다.

1. **평가 재현성 확보:** 실행 환경과 평가 경로를 고정하고, 같은 정책의 반복 평가 결과를 확인한다.
2. **조건별 반복 실험:** 복수의 학습 시드와 독립 평가 시나리오를 사용하여 설정 변경의 효과를 검증한다.
3. **학습 구조 점검:** 보상과 작업 성과의 정합성, 가치 추정 오차, 기본 정책의 행동 체크 정보 부족을 점검한다.

4. **실물 적용 검증:** 시뮬레이션에서 개선이 확인된 잔차 정책을 대상으로 실물에서도 효과가 유지되는지 평가한다.

4.5 최종 실물 추론 설정 탐색

4.2절의 데이터 구성 비교와 4.3절의 정책 비교 결과를 바탕으로, Pick & Place의 최종 평가 모델은 **실물·시뮬레이션 혼합 데이터로 학습한 ACT 정책**으로 선정하였다. 본 절과 다음 절에서는 선정한 모델을 실물 로봇에 적용하여 추론 하이퍼파라미터를 탐색하고, 최종 설정에서 반복 평가한 작업 성공 수준과 주요 실패 양상을 분석한다.

아울러 실물 데이터만으로 학습한 ACT 정책의 Pick & Stack 초기 평가 결과를 제시하여, 파지와 적재 단계의 수행 특성을 살펴본다.

aggregate_fn을 포함한 추론 설정에 따라 실제 동작의 매끄러움과 성공률이 달라진다. 실물 배포에서는 이를 바탕으로 여러 조합을 정성적으로 재비교하였다.

표 4-5 실물 추론 설정별 예비 관찰

chunk	threshold	aggregate_fn	관찰
60	0.4	conservative	2개까지는 순조로우나 이후 멈칫 거림
60	0.5	conservative	4개, 3개 (2회 시행)
60	0.5	latest_only	4개까지 매끄러움. 시행 간 편차 존재(5개 성공 사례도 있었음)
60	0.6	conservative	3개까지는 매끄러웠으나 이후 불안정
60	0.75	latest_only	소수 시행에서 5개까지 매끄럽게 이어지는 사례를 확인, 다른 조합 대비 후속 평가 후보
70	0.5	conservative	파지 위치를 빗나가는 동작이 잦았으나 3개는 옮겨냄
70	0.6	conservative	3개 성공

소수 시행 수준의 정성적 비교에서 chunk=60 / threshold=0.75 / aggregate_fn=latest_only 조합을 후속 평가 후보로 판단하여 이를 **최종 평가 설정**으로 채택하고, 이 설정으로 4.6절의 정식 반복 평가를 진행하였다.

4.6 최종 실물 평가 결과

4.6.1 Pick & Place 반복 평가

1. 정책: ACT
2. 데이터셋: so101_cube_place_dirty300sim_real_0908_1170 — Real 170개 + 완전 무작위 배치 Sim 1,000개(DRprod, dirty data). 총 1,170 episode

표 4-6 Pick & Place 20회 개별 평가 기록

시행	성공 개수	시행	성공 개수
1	4	11	2
2	2	12	4
3	2	13	2
4	4	14	4
5	5	15	3
6	5	16	5
7	3	17	3
8	2	18	1
9	4	19	5
10	5	20	3

3. 총 20회 평균 **3.4 / 5개 (68%)**
4. 5개 전부 성공: **5회 / 20회 (25%)**, 소요시간이 기록된 4건의 평균 **약 2분 29초**(2:40 / 2:47 / 1:40 / 2:50)
5. 완전 성공은 대체로 큐브 간 간격이 넓고 구역별로 한 개씩 고르게 배치되었을 때 나타났다. 다만 학습에 사용한 시뮬레이션 데이터는 블록 위치를 완전 무작위로 배치해 대량으로 확보하였음에도 이 경향이 재현되는 정확한 원인은 규명하지 못했으며, 추가 분석이 필요한 부분으로 남겨둔다.
6. 실패는 크게 두 유형에 집중되었다: ① 여러 번의 롤아웃에서 특정 위치에 놓인 블록을 반복적으로 잘 잡지 못하는 경우, ② 먼저 시도한 큐브를 잡지 못해 이후 시퀀스가 전반적으로 무너지는 경우.

4.6.2 Pick & Stack 초기 평가

1. 정책: ACT
2. 데이터셋: act_policy_so101_cube_stack_real_0908 — Real 100개
3. Stack은 아직 본격적으로 시도한 단계가 아니며, 현재는 실물 데이터만 수집한 상태에서 진행한 초기 평가다.

표 4-7 Pick & Stack 5회 초기 평가 기록

시행	1	2	3	4	5
결과	2층	5층*	1층	0층	1층

4. 5층을 기록한 두 번째 시행에서는 블록 1개의 파지를 사람이 보조하였다. 따라서 이 시행을 완전 자율 5층 적재 성공으로 집계하지 않았다.

5. 평균 1.8층(사람 보조 시행 포함)

6. "1층"은 블록을 한 번 옮기긴 했으나 실제로 쌓지는 못한 경우를 의미한다.
7. 핵심 관찰: 관찰된 시행에서는 파지가 주요 취약 단계였으며, 보조가 포함된 5층 시행에서는 파지 이후의 적재가 유지되었다. 5층까지 쌓는 동안 적재 실패(붕괴)는 한 번도 발생하지 않았으며, 이는 파지 단계의 우선 개선 필요성을 시사한다. 다만 5회에 불과한 초기 평가이므로 적재 안정성 전반을 확정하기에는 추가 검증이 필요하다.

5 결론 및 향후 연구 방향

5.1 성과

실물 환경에서는 Jetson Orin Nano의 RobotClient와 RTX 4090 서버의 PolicyServer를 연결하는 비동기 추론 파이프라인을 구축하였다. 초기에는 Tailscale을 경유하면서 통신 지연이 크게 발생했으나, 이후 직접 통신 경로를 구성하고 이미지 전송에 **JPEG 압축**을 적용하여 지연을 줄였다.

정책 실행 중 실패 상황을 추가로 수집하기 위해 Human-in-the-Loop DAgger 방식도 적용하였다. 사람이 개입해야 하는 시점에만 제어권을 넘겨 복구 동작을 수행하도록 하여, 기존 정책의 실행 흐름을 유지한 상태에서 실패 이후의 복구 구간까지 연속된 에피소드로 수집할 수 있도록 하였다.

시뮬레이션 환경에서는 총 179개의 랜덤화 축을 **12개의 gate**로 구분하여 도메인 랜덤화 파이프라인을 구축하였다. 해당 환경에서 IK 기반 교사는 Pick & Place 기준 0.933의 성공률을 기록하였다.

성공 기준을 만족한 rollout만 선별하는 방식으로 1,000 episode 규모의 시뮬레이션 데이터셋을 생성하였으며, 1,000 episode를 약 3~4시간 내에 생성한 기록도 확보하였다.

또한 **시뮬레이션 1,000 episode와 실물 170 episode**를 함께 사용해 학습한 정책을 실물 환경에 적용했을 때 블록을 성공적으로 옮기는 사례를 확인하였다. 이를 통해 도메인 랜덤화를 적용한 시뮬레이션 데이터가 sim-to-real gap을 완화하는 데 일정 부분 기여한 것으로 판단하였다. 다만 해당 효과를 정량적으로 측정한 것은 아니며, 실제 성공 사례에 근거한 관찰 결과이다.

최종 설정인 chunk=60, threshold=0.75, latest_only 조건에서 Pick & Place는 **20회 평가 기준 평균 3.4/5개**, 블록 배치 비율 68%를 기록하였고, 5개 블록을 모두 옮긴 **완전 성공률은 25%**였다. 완전 성공 5회 중 시간이 기록된 4회의 평균 수행 시간은 149.25초(약 2분 29초)였다.

Pick & Stack은 **사람의 파지 보조**가 포함된 5회 기록에서 평균 1.8층을 기록하였다. 이 값은 완전 자율 적재 성능을 뜻하지 않는다.

두 미션 모두에서 주요 실패는 배치나 적재 단계보다 파지 단계에서 더 많이 발생했으며, 최종 성능의 주된 병목이 **파지 안정성**에 있음을 확인하였다.

5.2 미완료 항목과 후속 개발

(1) Task-conditioned 멀티태스크 정책

규정상 Pick & Place와 Pick & Stack을 서로 다른 신경망으로 분리할 수 없기 때문에, 하나의 정책이 두 미션을 모두 수행할 수 있어야 한다. 현재 사용 중인 ACT는 텍스트 인코더를 사용하지 않아 --task=로 전달한 문자열이 모델 입력에 직접 반영되지 않는다.

이에 task 정보를 one-hot 벡터로 변환해 observation.state에 추가하는 방식을 적용하였다. 기존 6차원 관절 상태에 task_id_0, task_id_1을 추가하여 입력을 **8차원으로 확장**하였다.

초기에는 environment_state를 활용하는 방식도 시도했으나, 실물 로봇 경로에서 해당 입력을 정상적으로 처리하지 못해 현재 방식으로 변경하였다.

현재 Pick & Place 300 episode와 Pick & Stack 300 episode를 병합한 **8차원 데이터셋 so101_cube_multitask_task_merged 총 600 episode**를 확보하였으며, 신규 데이터 수집에 필요한 로봇 래퍼 클래스도 구현과 검증을 마쳤다. 다음 단계에서는 해당 데이터로 ACT를 학습한 뒤, 입력한 task_id에 따라 두 미션의 행동이 실제로 구분되는지 확인할 예정이다.

(2) Pick & Stack 데이터 수율

시뮬레이션 도메인 랜덤화 조건에서 Pick & Stack 데이터의 채택률은 21.7~40.0%로 Pick & Place보다 낮게 나타났다. 여러 환경 변수를 조정하며 원인을 확인하고 있으나, 현재까지 수율을 결정하는 뚜렷한 요인은 특정하지 못했다.

또한 Pick & Stack은 아직 시뮬레이션 데이터와 실물 데이터를 합쳐 학습한 뒤 실물 rollout까지 평가하지 못한 상태이다. 이후 데이터 생성 수율을 개선하고 충분한 데이터를 확보한 뒤, 실물 데이터와 시뮬레이션 데이터를 함께 사용한 학습 및 평가를 진행할 예정이다.

(3) 파지 성능 보강

4.6절의 평가 결과에서 두 미션 모두 주요 병목이 파지 단계에 있음을 확인하였다. 이를 보완하기 위해 파지 동작에 집중한 별도 데이터를 추가로 수집할 계획이다. 블록의 시작 위치를 가우시안 분포로 변화시키고, 블록을 일정 높이 이상 들어 올리면 에피소드를 종료하는 방식으로 파지 중심 데이터를 구성한다. 생성된 데이터는 Pick & Place와 Pick & Stack 학습에 공통으로 활용할 예정이다.

(4) Residual RL

Pick & Place의 성공률을 높이기 위한 Residual RL 연구도 병행하고 있다. 4장에서 수행한 보상 설계, 잔차 행동 범위, base policy 선택 등의 실험 결과를 바탕으로 현재 추가 학습과 파라미터 조정을 진행하고 있다.

5.3 향후 연구 방향

(1) 2차 미션 확장과 단일 정책 통합

Pick & Stack 시연 데이터를 추가로 확보하고, 현재 구현한 task-conditioned 구조를 실제 학습에 적용할 예정이다. 시뮬레이션에서 생성한 교사 데이터와 사람이 감독한 tele-operation 데이터를 함께 사용하여 하나의 정책이 Pick & Place와 Pick & Stack을 모두 수행할 수 있는지 검증한다.

(2) Residual RL 보상 설계 및 하이퍼파라미터 개선

현재 Residual RL에서는 안정적인 성능 개선이 확인되지 않았으며, 보상과 평가 지표의 정합성 및 가치 추정 오차에 대한 점검이 필요하다. 이를 보완하기 위해 dense reward 구성을 조정하고, 보상 해킹이 발생하지 않도록 보상 항목 간 균형을 다시 설계할 예정이다. 또한 exploration noise와 잔차 행동 범위(팔 관절 $\pm 3^\circ$, 그리퍼 $\pm 5^\circ$) 등 주요 하이퍼파라미터를 조정하여 학습 안정성과 보정 성능을 높이고자 한다.

(3) 대규모 사전학습 모델 기반 정책으로의 확장

장기적으로는 현재의 ACT 기반 정책을 VLA 계열 모델로 확장하여 언어 지시를 활용한 범용 조작까지 적용 범위를 넓힐 수 있다. 현재 데이터 규모에서는 SmoIVLA보다 ACT의 성능이 높게 나타났으므로, 우선 학습 데이터 규모를 확대한 뒤 두 모델을 다시 비교하는 방향으로 진행할 예정이다.

6 과제 수행 관리

6.1 구성원별 역할과 수행 결과

표 6-1 구성원별 역할과 수행 내용

구성원	주요 담당 분야	수행 내용
여채언	정책 모델 학습 및 강화 학습 설계	ACT-SmolVLA 학습과 성공률·추론 성능 비교, 최종 적용 정책 선정, 잔차 강화학습 보상 설계와 학습 실행, 경계 있는 국소 RL 게이트 설계
옥소미	데이터 수집 및 실험 설계, 강화학습 설계	하드웨어 셋업과 데이터 수집 절차 정립, Tele-operation 시연 데이터 및 HIL 개입 데이터 수집, 카메라 배치 실험, 잔차 강화학습 보상 설계와 학습 실행
문여원	시뮬레이션 환경 구축 및 시스템 통합	MuJoCo 경기장·판정 구현, IK 기반 교사 정책 개발, 도메인 랜덤화 179축 설계·구현, 시뮬 데이터 생성 및 LeRobot 변환, Sim-to-real 정합 계측

6.2 개발 일정과 계획 대비 수행 결과

착수보고서는 5월부터 9월까지의 월별 계획을 제시했고, 중간보고서는 7~9월 계획을 시뮬레이션·강화학습 검토와 최종 안정화 중심으로 갱신하였다. 최종 수행 결과는 다음과 같다.

표 6-2 월별 개발 계획과 최종 수행 결과

기간	착수 계획	중간 조정 및 최종 수행 결과
5월	SO-101 환경 구축, Place 시연 수집, SmolVLA 초기 학습	중간 보고 시점에 하드웨어·서버 연결과 Place 100개 시연 확보
6월	Stack 시연 수집과 초기 정책 평가	중간보고에서 ACT-Stack 50개 기반 카메라 비교를 수행. 최종 원고는 Real 100개 기반 Stack 초기 평가를 보고
7월	시뮬레이션 데이터 생성과 Sim-to-Real 비교	중간 계획을 MuJoCo 환경, 시각·동역학 랜덤화, RL 적용 가능성 검토로 구체화. 최종적으로 IK 교사와 랜덤화 데이터 파이프라인 구축
8월	강화학습 보상 설계 및 정책 개선	배치 분포 확대, 보상 결함 수정, 잔차 탐색·반복 학습 수행. RL의 안정적 성능 개선은 미확인

기간	작수 계획	중간 조정 및 최종 수행 결과
9월	안정화, 성능 비교, 보고서 작성	혼합 데이터 ACT 선정, 추론 설정 탐색, Place 20회와 Stack 5회 실물 평가. 단일 정책 통합은 후속 과제로 남음

시스템과 데이터 생성 기반은 구축하였으나, 완전 자율 적재의 반복 검증, **단일 모델 통합 성능 검증, 잔차 강화학습의 안정적 개선**은 남은 작업이다.

7 참고문헌

- [1] Hugging Face, "ACT Action Chunking with Transformers," LeRobot Documentation. [Online]. Available: <https://huggingface.co/docs/lerobot/act> (accessed Sep. 12, 2026).
- [2] Hugging Face, "SmolVLA," LeRobot Documentation. [Online]. Available: <https://huggingface.co/docs/lerobot/smolvla> (accessed Sep. 12, 2026).
- [3] Hugging Face, "Asynchronous Inference," LeRobot Documentation. [Online]. Available: <https://huggingface.co/docs/lerobot/async> (accessed Sep. 12, 2026).
- [4] Google DeepMind, "Overview," MuJoCo Documentation. [Online]. Available: <https://mujoco.readthedocs.io/en/stable/overview.html> (accessed Sep. 12, 2026).
- [5] L. Ankile, Z. Jiang, R. Duan, G. Shi, P. Abbeel, and A. Nagabandi, "Residual Off-Policy RL for Finetuning Behavior Cloning Policies," arXiv:2509.19301, 2025. [Online]. Available: <https://arxiv.org/html/2509.19301v1>.
- [6] G. Ma, L. Li, H. Wang, Z. Liu, P.-L. Bacon, and D. Tao, "What Makes Value Learning Efficient in Residual Reinforcement Learning?," arXiv:2602.10539, 2026. [Online]. Available: <https://arxiv.org/abs/2602.10539>.